

Day02-文件管理帮助系统文本编辑用户管理

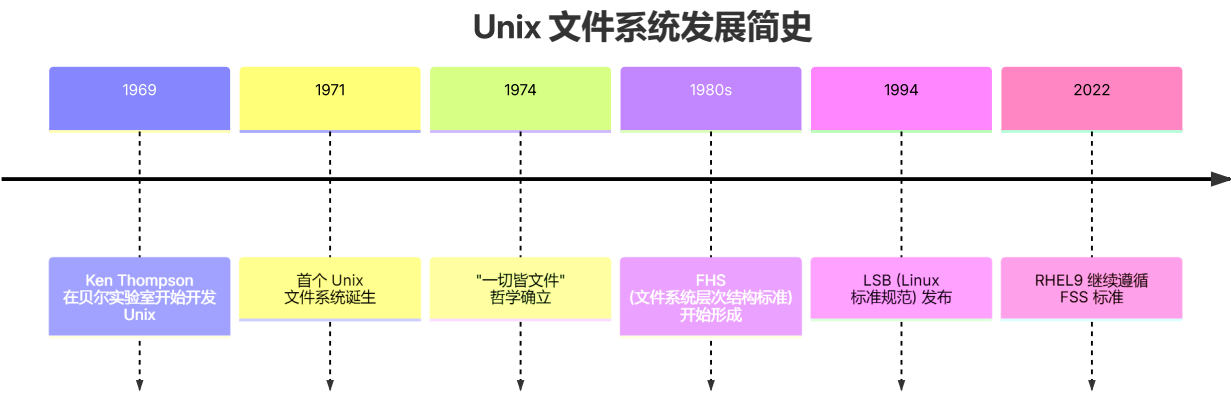
课程目标

- 掌握文件系统层次结构和路径导航
- 学会使用命令行管理文件和目录
- 掌握硬链接和软链接的使用
- 学会使用本地帮助系统
- 掌握重定向和管道操作
- 学会使用vim编辑器
- 掌握用户和组管理

课前故事：Unix文件系统的诞生

在1970年代初期，Unix的创造者Ken Thompson和Dennis Ritchie在设计Unix文件系统时，引入了一个革命性的概念——一切皆文件（Everything is a File）。

设计哲学的诞生：



为什么要统一文件层次结构？

想象一下，如果你每换一台电脑都要重新学习文件存放在哪里，那将是多么痛苦的事情。FHS（Filesystem Hierarchy Standard，文件系统层次结构标准）的诞生解决了这个问题：

- 一致性：无论使用哪个Linux发行版，核心目录结构都是一样的
- 可预测性：系统管理员可以准确知道配置文件、日志、程序的位置
- 协作性：不同开发者开发的软件可以遵循相同的规范

这种设计思想一直延续到今天，成为了Linux系统的核心特征之一。

上午 - 文件管理与帮助系统

第一部分：深入理解文件系统层次结构

1.1 重要目录深度解析

核心目录（必须掌握）：

目录	用途	重要	使用场景
/etc	配置文件	☆☆☆	修改网络、SSH、服务配置
/var/log	日志文件	☆☆☆	排查SELinux问题、查看错误
/home	用户家目录	☆☆☆	设置协作目录、权限
/tmp	临时文件	☆☆	存放临时数据
/usr/bin	用户命令	☆	创建脚本链接
/root	root家目录	☆☆☆	创建管理脚本

目录结构可视化：

Linux Filesystem Hierarchy - Key Directories

找不到"assets/day02-filessystem-hierarchy.excalidraw"。

Important Directory Descriptions:

- **/etc** - All configuration files (network, services, SSH)
- **/home** - Regular user home directories
- **/root** - Administrator home directory
- **/var/log** - System logs for troubleshooting
- **/tmp** - Temporary files (auto-cleaned after 10 days)

RHEL7+的重要变化（重点）：

为了简化系统维护，RHEL7开始进行了目录合并：

原目录	合并到	说明
/bin	/usr/bin	用户命令
/sbin	/usr/sbin	系统管理命令
/lib	/usr/lib	32位库文件
/lib64	/usr/lib64	64位库文件

⚠ 注意： 这些目录现在都是符号链接，不是真实目录！

SHELL

```
# 查看链接关系
ls -l /bin
# lrwxrwxrwx. 1 root root 7 Jan 15 10:00 /bin -> usr/bin

ls -l /sbin
# lrwxrwxrwx. 1 root root 8 Jan 15 10:00 /sbin -> usr/sbin
```

1.2 路径的深度理解

绝对路径 vs 相对路径（生活类比）：

想象你在一个大型商场里问路：

绝对路径就像："从商场正门进入 → 上一楼 → 直走100米 → 左转 → 第3家店"

- 总是从固定起点（根目录 /）开始
- 无论你在哪里，都能准确找到目标
- 示例： `/home/student/documents/file.txt`

相对路径就像： "从你现在的位置直走50米 → 右转"

- 从当前位置开始计算
- 更简洁但依赖当前位置
- 示例： `documents/file.txt` (从/home/student出发)

路径命令详解：

SHELL

```
# pwd - Print Working Directory (显示当前工作目录)
[root@node1 ~]# pwd
/root

[root@node1 ~]# cd /home/student
[root@node1 student]# pwd
/home/student

# 输出说明：显示完整的绝对路径
```

特殊路径符号速查表：

符号	含义	等价路径	使用场景
.	当前目录	<code>/home/student</code>	明确指定当前目录
..	父目录	<code>/home</code> (从student出发)	返回上一级
~	当前用户家目录	<code>/root</code> (root用户)	快速返回家目录
<code>~username</code>	指定用户家目录	<code>/home/student</code>	访问其他用户家目录
-	上一个工作目录	切换前的目录	在两个目录间切换

路径切换实战示例：

SHELL

```
# 示例1: 使用cd在目录间导航
[root@node1 ~]# pwd
/root
[root@node1 ~]# cd /etc/sysconfig/network-scripts/
[root@node1 network-scripts]# pwd
/etc/sysconfig/network-scripts

# 使用 - 返回上一个目录
[root@node1 network-scripts]# cd -
/root

# 示例2: 使用 ~ 快速返回家目录
[root@node1 network-scripts]# cd ~
[root@node1 ~]# pwd
/root

# 示例3: 相对路径 vs 绝对路径
[root@node1 ~]# cd /usr/local/bin
[root@node1 bin]# cd ../../share/man
[root@node1 man]# pwd
/usr/share/man

# 使用绝对路径直接到达
[root@node1 man]# cd /usr/share/man
[root@node1 man]# pwd
/usr/share/man
```

⚠ 易错点:

- 1. 路径必须完全一致，建议复制粘贴
- 2. 区分大小写: `File.txt` ≠ `file.txt`
- 3. 注意空格和特殊字符，使用引号或转义

教材练习

练习名称	对应章节	说明
选择练习: DESCRIBING LINUX FILE SYSTEM HIERARCHY CONCEPTS	第03章 第01节	Linux 文件系统层次结构概念

第二部分：文件管理命令深度解析

2.1 复制文件和目录 - cp命令

cp命令的强大功能:

SHELL

```
# 基本复制: 复制单个文件
cp /etc/hosts /tmp/

# 参数说明:
# cp = copy, 复制命令
# /etc/hosts = 源文件 (绝对路径)
# /tmp/ = 目标目录

# 完整输出示例
[root@node1 ~]# cp /etc/hosts /tmp/
[root@node1 ~]# ls -l /tmp/hosts
-rw-r--r--. 1 root root 198 Jan 15 10:30 /tmp/hosts

# 输出说明:
# -rw-r--r-- = 文件权限 (644)
# 1 = 硬链接数量
# root root = 文件所有者和所属组
# 198 = 文件大小 (字节)
# Jan 15 10:30 = 最后修改时间
```

cp命令选项详解 (常用):

选项	全称	作用	使用场景
-r	recursive	递归复制目录	复制整个目录树
-a	archive	归档模式, 保留所有属性	备份文件
-p	preserve	保留时间戳和权限	保持文件元数据
-v	verbose	显示复制过程	调试和确认
-i	interactive	覆盖前提示	安全操作
-f	force	强制覆盖	危险操作, 慎用

⚠ 警告: Linux没有回收站! `cp -f` 会直接覆盖, 无法恢复!

递归复制目录：

SHELL

```
# 复制整个yum配置目录
cp -r /etc/yum /tmp/

# 完整执行示例
[root@node1 ~]# cp -r /etc/yum /tmp/
[root@node1 ~]# ls -R /tmp/yum/
/tmp/yum/:
repos.d/

/tmp/yum/repos.d/:
redhat.repo

# 输出说明：
# -r 选项递归复制了整个yum目录及其子目录
```

归档模式备份（推荐）：

SHELL

```
# cp -a 等价于 -dR --preserve=all
cp -a /etc/hosts /tmp/hosts-backup

# 参数详解：
# -d = --no-dereference, 不跟随符号链接
# -R = --recursive, 递归
# --preserve=all = 保留所有属性（权限、时间戳、所有权、链接等）

# 实际应用示例
[root@node1 ~]# cp -a /etc/hosts /tmp/hosts-backup
[root@node1 ~]# stat /etc/hosts /tmp/hosts-backup | grep -E "File:|Modify:"
File: /etc/hosts
Modify: 2025-01-15 10:00:00.000000000 +0800
File: /tmp/hosts-backup
Modify: 2025-01-15 10:00:00.000000000 +0800

# 说明：时间戳完全一致，所有属性都保留了
```

复制多个文件（效率技巧）：

SHELL

```
# 方法1：同时复制多个文件到同一目录
cp /etc/hosts /etc/hostname /etc/resolv.conf /tmp/

# 方法2：使用通配符 (wildcard)
cp /etc/*.conf /tmp/

# 方法3：使用大括号扩展 (Brace Expansion)
cp /etc/hosts{,.bak}
# 等价于：cp /etc/hosts /etc/hosts.bak

# 实际应用示例
[root@node1 ~]# cp /etc/hosts /etc/hostname /tmp/
[root@node1 ~]# ls /tmp/{hosts,hostname}
/tmp/hosts /tmp/hostname
```

2.2 移动和重命名 - mv命令

mv的双重身份：

mv命令既是"移动"命令，也是"重命名"命令：

- 跨目录移动 = 移动文件
- 同目录操作 = 重命名文件

移动文件:

SHELL

```
# 将文件移动到其他目录
mv /tmp/hosts /tmp/backup/

# 完整执行示例
[root@node1 ~]# touch /tmp/testfile
[root@node1 ~]# mkdir /tmp/backup
[root@node1 ~]# mv /tmp/testfile /tmp/backup/
[root@node1 ~]# ls /tmp/backup/
testfile

# 说明: 文件从/tmp/移动到了/tmp/backup/
```

重命名文件:

SHELL

```
# 重命名文件 (同一目录内)
mv /tmp/backup/hosts /tmp/backup/hosts.old

# 完整执行示例
[root@node1 ~]# ls /tmp/backup/
hosts
[root@node1 ~]# mv /tmp/backup/hosts /tmp/backup/hosts.old
[root@node1 ~]# ls /tmp/backup/
hosts.old

# 说明: 文件名从hosts改为hosts.old
```

同时移动多个文件 (效率技巧) :

SHELL

```
# 方法1: 移动多个文件到一个目录
mv file1.txt file2.txt file3.txt /tmp/backup/

# 方法2: 使用通配符
mv *.log /tmp/logs/

# 方法3: 移动目录
mv old_directory/ new_directory/

# 实际应用示例
[root@node1 ~]# mkdir -p /tmp/docs
[root@node1 ~]# touch /tmp/file{1..5}.txt
[root@node1 ~]# mv /tmp/file*.txt /tmp/docs/
[root@node1 ~]# ls /tmp/docs/
file1.txt file2.txt file3.txt file4.txt file5.txt
```

⚠ 易错点:

1. `mv` 覆盖同名文件时默认不提示! 使用 `mv -i` 更安全
2. 移动目录时, 目标目录不能已存在
3. 跨文件系统移动实际上是"复制+删除", 时间较长

2.3 删除文件和目录 - rm命令

rm命令的威力与危险:

```
# 删除单个文件
rm /tmp/file.txt

# 删除前会提示（默认）
[root@node1 ~]# rm /tmp/file.txt
rm: remove regular empty file '/tmp/file.txt'? y

# 输入 y 确认删除, n 取消删除
```

⚠️ 致命警告:

```
# ⚠️ 绝对不要在生产环境执行以下命令！
rm -rf /
rm -rf /*
rm -rf /etc
rm -rf ~/*

# 这些命令会：
# -r = 递归删除所有子目录
# -f = 强制删除，不提示确认
# / = 从根目录开始（整个系统）
# 结果 = 系统完全损坏，无法恢复！
```

安全删除技巧:

```
# 使用 -i 选项进行交互式删除（推荐）
rm -i /tmp/file.txt

# 使用 -I 选项，删除超过3个文件时才提示
rm -I /tmp/*.log

# 查看要删除什么，但不真的删除（模拟）
rm --dry-run -rf /tmp/test/
# 或使用 echo 先确认
echo /tmp/test/* | xargs rm
```

递归删除目录:

```
# 删除目录及其内容
rm -r /tmp/backup/

# 使用 -rf 强制删除（谨慎使用）
rm -rf /tmp/old_data/

# 完整执行示例
[root@node1 ~]# mkdir -p /tmp/test/{a,b,c}
[root@node1 ~]# rm -r /tmp/test/
rm: descend into directory '/tmp/test'? y
rm: remove directory '/tmp/test/a'? y
rm: remove directory '/tmp/test/b'? y
rm: remove directory '/tmp/test/c'? y
rm: remove directory '/tmp/test'? y

# 说明: -r会递归删除，对每个目录和文件都提示
```

删除任务的正确做法:

```
# 任务：删除指定目录但保留其他文件
# X 错误做法（危险）
rm -rf /tmp/*

# ✅ 正确做法（精确指定）
rm -rf /tmp/specific_directory

# 验证后再执行
ls -ld /tmp/specific_directory
rm -rf /tmp/specific_directory
```

rmdir - 仅删除空目录：

```
# 只能删除空目录
mkdir /tmp/empty
rmdir /tmp/empty # 成功

mkdir /tmp/not_empty
touch /tmp/not_empty/file.txt
rmdir /tmp/not_empty # 失败
# rmdir: failed to remove '/tmp/not_empty': Directory not empty

# 强制删除非空目录（不推荐）
rm -rf /tmp/not_empty
```

2.4 mkdir - 创建目录

基本创建：

```
# 创建单个目录
mkdir /tmp/test

# 创建多个目录
mkdir /tmp/dir1 /tmp/dir2 /tmp/dir3

# 完整执行示例
[root@node1 ~]# mkdir /tmp/test
[root@node1 ~]# ls -ld /tmp/test
drwxr-xr-x. 2 root root 6 Jan 15 11:00 /tmp/test

# 输出说明：
# drwxr-xr-x = 目录权限（755）
# 2 = 硬链接数（. 和 ..）
# root root = 所有者和所属组
# 6 = 目录大小（只是目录项本身）
```

递归创建（**-p**选项）：


```
# 创建多级目录结构
mkdir -p /tmp/project/{src,doc,test}/{main,backup}

# 展开后的目录结构:
# /tmp/project/
# |— src/
# |   |— main/
# |   └─ backup/
# |— doc/
# |   |— main/
# |   └─ backup/
# └─ test/
#     |— main/
#     └─ backup/

# 完整执行示例
[root@node1 ~]# mkdir -p /tmp/project/{src,doc,test}
[root@node1 ~]# tree /tmp/project/
/tmp/project/
├─ doc
├─ src
└─ test

3 directories, 0 files
```

设置权限同时创建:

```
# 创建目录时指定权限
mkdir -m 750 /tmp/securedir

# 创建多个目录并设置权限
mkdir -m 755 /tmp/public /tmp/shared

# 完整执行示例
[root@node1 ~]# mkdir -m 750 /tmp/securedir
[root@node1 ~]# ls -ld /tmp/securedir
drwxr-x---. 2 root root 6 Jan 15 11:05 /tmp/securedir

# 权限说明: 750 = rwxr-x---
# 所有者: 读写执行
# 组: 读执行
# 其他: 无权限
```

练习名称	对应章节	说明
练习: MANAGING FILES USING COMMAND-LINE TOOLS	第03章 第03节	使用命令行工具管理文件

第三部分：硬链接和软链接深度解析

3.1 硬链接 - ln命令

什么是硬链接？（必须掌握的概念）

硬链接是文件的另一个名称，指向同一个inode（索引节点）。在Linux系统中，文件名不是文件的本质，**inode**才是！

硬链接的核心特性：

找不到“assets/day02-hardlinks.excalidraw”。

硬链接的4大特性：

1. 完全相同：所有硬链接指向同一个数据块，没有任何主次之分
2. 同步更新：修改任何一个硬链接，所有其他链接都会看到变化
3. 独立存在：删除原文件，硬链接仍然有效
4. 同文件系统限制：只能在同一文件系统内创建

创建和查看硬链接：

```
# 创建硬链接 (不加 -s 选项)
ln /etc/hosts /tmp/hosts-hardlink

# 完整执行示例
[root@node1 ~]# touch /tmp/original.txt
[root@node1 ~]# echo "Hello RHCSA" > /tmp/original.txt
[root@node1 ~]# ln /tmp/original.txt /tmp/hardlink1.txt
[root@node1 ~]# ln /tmp/original.txt /tmp/hardlink2.txt

# 查看硬链接关系 (关键: -i 选项查看inode)
[root@node1 ~]# ls -li /tmp/*.txt
262151 -rw-r--r--. 3 root root 12 Jan 15 11:30 /tmp/hardlink1.txt
262151 -rw-r--r--. 3 root root 12 Jan 15 11:30 /tmp/hardlink2.txt
262151 -rw-r--r--. 3 root root 12 Jan 15 11:30 /tmp/original.txt

# 输出详解:
# 262151 = inode号 (三个文件完全相同!)
# 3 = 硬链接数量 (有三个文件名指向这个inode)
# -rw-r--r-- = 文件权限
# 12字节 = 文件大小
```

SHELL

硬链接的独立性证明：

```
# 删除原文件，硬链接仍然有效
[root@node1 ~]# rm /tmp/original.txt
[root@node1 ~]# cat /tmp/hardlink1.txt
Hello RHCSA

# 查看硬链接数量减少
[root@node1 ~]# ls -li /tmp/hardlink*.txt
262151 -rw-r--r--. 2 root root 12 Jan 15 11:30 /tmp/hardlink1.txt
262151 -rw-r--r--. 2 root root 12 Jan 15 11:30 /tmp/hardlink2.txt

# 说明：硬链接数从3变成了2，但数据仍然完整
```

SHELL

实际场景：协作目录备份

```
# 场景：多个用户需要访问同一个配置文件
ln /etc/app.conf /home/user1/app.conf
ln /etc/app.conf /home/user2/app.conf

# 任何一个用户修改文件，其他用户都能看到
echo "timeout=60" >> /home/user1/app.conf
cat /home/user2/app.conf # 可以看到新添加的内容
```

SHELL

⚠ 硬链接的限制（重点）：

```
# X 不能为目录创建硬链接
ln /tmp/testdir /tmp/testdir-hardlink
# ln: /tmp/testdir: hard link not allowed for directory

# X 不能跨文件系统创建硬链接
ln /etc/hosts /boot/hosts-hardlink
# ln: failed to create hard link '/boot/hosts-hardlink' => '/etc/hosts': Invalid cross-device link

# ✅ 软链接可以做到以上两点 (见下一节)
```

3.2 软链接（符号链接） - ln -s命令

什么是软链接？（必须掌握）

软链接（Symbolic Link，又称符号链接）是一个特殊的文件，其中包含的是另一个文件的路径。类似于Windows的快捷方式。

软链接 vs 硬链接对比（重要表格）：

特性	硬链接	软链接
命令	ln file link	ln -s file link
inode	相同	不同
跨文件系统	❌ 不支持	✅ 支持
链接目录	❌ 不支持	✅ 支持
删除原文件	✅ 仍有效	❌ 变成悬挂链接
权限	与原文件相同	rw-rw-rw-（全是执行权限）
文件类型	普通文件 (-)	符号链接 (l)

创建软链接：

```
# 创建软链接 (-s 选项)
ln -s /etc/hosts /tmp/hosts-symlink

# 完整执行示例
[root@node1 ~]# ln -s /etc/hosts /tmp/hosts-link
[root@node1 ~]# ls -l /tmp/hosts-link
lrwxrwxrwx. 1 root root 10 Jan 15 12:00 /tmp/hosts-link -> /etc/hosts

# 输出详解：
# l = 符号链接文件类型
# lrwxrwxrwx = 软链接权限（固定显示，实际权限由原文件决定）
# -> /etc/hosts = 指向的目标路径
```

软链接的相对路径陷阱（易错点！）：

```
# X 错误示范：使用相对路径创建软链接
cd /tmp
ln -s ../etc/hosts hosts-link
# 问题：如果从其他目录访问，路径会失效

# ✔ 正确做法：使用绝对路径
ln -s /etc/hosts /tmp/hosts-link
ln -s ../../etc/hosts /tmp/very/deep/path/hosts-link # 相对路径也能工作，但容易出错

# 完整示例说明错误路径的问题
[root@node1 ~]# cd /tmp
[root@node1 tmp]# ln -s ../etc/hosts bad-link
[root@node1 tmp]# cat bad-link # 从/tmp访问正常
127.0.0.1 localhost

[root@node1 tmp]# cd /root
[root@node1 ~]# cat /tmp/bad-link # 从/root访问失败
cat: /tmp/bad-link: No such file or directory
# 原因：相对路径 ../etc/hosts 从/root/tmp/解析，路径不存在
```

软链接指向目录（常见）：

```
# 创建目录软链接
ln -s /var/log /tmp/logs

# 完整执行示例
[root@node1 ~]# ln -s /var/log /tmp/logs
[root@node1 ~]# ls -ld /tmp/logs
lrwxrwxrwx. 1 root root 8 Jan 15 12:10 /tmp/logs -> /var/log
[root@node1 ~]# ls /tmp/logs/ | head -5
anaconda/
boot.log
dmesg
```

悬挂软链接（Dangling Symlink）：

```
# 删除原文件后的软链接状态
[root@node1 ~]# rm /etc/hosts
[root@node1 ~]# ls -l /tmp/hosts-link
lrwxrwxrwx. 1 root root 10 Jan 15 12:00 /tmp/hosts-link -> /etc/hosts
[root@node1 ~]# cat /tmp/hosts-link
cat: /tmp/hosts-link: No such file or directory

# 说明：软链接仍然存在，但指向的目标已经不存在
```

实际场景：配置文件链接管理

```
# 任务：为配置文件创建软链接以便于访问
# 常见需求：在用户家目录创建配置文件链接

ln -s /etc/app.conf ~user/app-config
ln -s /var/log/app.log ~user/app-log

# 验证链接
ls -l ~user/app-config
ls -l ~user/app-log
```

3.3 链接管理高级技巧

查看链接信息：

```
# readlink - 显示符号链接的目标
[root@node1 ~]# readlink /tmp/hosts-link
/etc/hosts

# readlink -f - 解析所有符号链接，显示最终目标
[root@node1 ~]# ln -s /tmp/hosts-link /tmp/link2link
[root@node1 ~]# readlink /tmp/link2link
/tmp/hosts-link
[root@node1 ~]# readlink -f /tmp/link2link
/etc/hosts

# find - 查找所有符号链接
[root@node1 ~]# find /tmp -type l
/tmp/hosts-link
/tmp/link2link

# find - 查找悬挂链接
[root@node1 ~]# find /tmp -xtype l
/tmp/broken-link
```

更新链接：

```
# 强制更新链接 (-f 覆盖已存在的链接)
ln -sf /new/path /tmp/link

# 使用 -n 更新链接而不是跟随链接
ln -sfn /new/path /tmp/link
```

练习

练习名称	对应章节	说明
练习：MAKING LINKS BETWEEN FILES	第03章 第04节	在文件之间创建链接

第四部分：Shell通配符与模式匹配

4.1 通配符的威力

通配符的故事：

在早期Unix系统中，管理员需要批量处理大量文件。手动输入每个文件名既繁琐又容易出错。于是，"通配符"（Wildcard）功能诞生了——它像一副扑克牌中的"万能牌"（Joker），可以匹配任意内容。

通配符对照表：

通配符	匹配规则	示例	常见用途
*	匹配0个或多个任意字符	*.conf	匹配所有配置文件
?	匹配单个任意字符	file?.txt	匹配file1.txt但不匹配file10.txt
[...]	匹配方括号内任一字符	file[123].txt	匹配特定文件
[^...]	匹配不在方括号内的字符	file[^0-9].txt	排除数字文件

通配符	匹配规则	示例	常见用途
[a-z]	匹配a到z的小写字母	[a-z]*.txt	匹配小写开头的文件
[A-Z]	匹配A到Z的大写字母	[A-Z]*.txt	匹配大写开头的文件
[0-9]	匹配0到9的数字	file[0-9].txt	匹配数字结尾的文件
{a,b,c}	匹配a或b或c	file.{txt,log}	大括号扩展

通配符实战示例（完全输出）：

SHELL

```
# 示例1: * 星号通配符
[root@node1 ~]# touch /tmp/file1.txt /tmp/file2.txt /tmp/file3.txt
[root@node1 ~]# ls /tmp/file*.txt
/tmp/file1.txt /tmp/file2.txt /tmp/file3.txt

# 示例2: ? 问号通配符
[root@node1 ~]# ls /tmp/file?.txt
/tmp/file1.txt /tmp/file2.txt /tmp/file3.txt
# 注意: file10.txt不会被匹配, 因为?只匹配单个字符

# 示例3: [] 字符集合
[root@node1 ~]# touch /tmp/fileA.txt /tmp/fileB.txt /tmp/fileC.txt
[root@node1 ~]# ls /tmp/file[ABC].txt
/tmp/fileA.txt /tmp/fileB.txt /tmp/fileC.txt

# 示例4: 范围匹配
[root@node1 ~]# ls /tmp/file[0-9].txt
/tmp/file1.txt /tmp/file2.txt /tmp/file3.txt

# 示例5: 反向匹配
[root@node1 ~]# ls /tmp/file[^0-9].txt
/tmp/fileA.txt /tmp/fileB.txt /tmp/fileC.txt
```

大括号扩展 (Brace Expansion)：

SHELL

```
# 生成序列
[root@node1 ~]# echo file{1..5}.txt
file1.txt file2.txt file3.txt file4.txt file5.txt

# 创建多个文件
[root@node1 ~]# touch test{a,b,c}.txt
[root@node1 ~]# ls test*.txt
testa.txt testb.txt testc.txt

# 嵌套扩展
[root@node1 ~]# echo {user1,user2}/{file1,file2}.txt
user1/file1.txt user1/file2.txt user2/file1.txt user2/file2.txt

# 实际应用: 批量创建目录
[root@node1 ~]# mkdir -p /project/{src,test,doc}/{main,backup}
[root@node1 ~]# tree /project/
/project/
├── doc
│   ├── backup
│   └── main
├── src
│   ├── backup
│   └── main
└── test
    ├── backup
    └── main
```

场景：查找特定文件

```
# 任务：查找所有.log文件
find /var/log -name "*.log"

# 任务：查找所有以ssh开头的配置文件
ls /etc/ssh/sshd_config /etc/ssh/ssh_config
# 或使用通配符
ls /etc/ssh/ssh*

# 任务：删除特定模式的文件
rm -f /tmp/temp[0-9].tmp
```

⚠ 易错点：

- 1. * 不匹配隐藏文件（以.开头的文件）
- 2. ? 只匹配单个字符，file10.txt不会被file?.txt匹配
- 3. 引号的使用：当模式包含空格或特殊字符时必须用引号

```
# X 错误
ls /tmp/my files/*.txt
# ✔ 正确
ls /tmp/my\ files/*.txt
ls "/tmp/my files/"*.txt
```

练习名称	对应章节	说明
选择练习：SPECIFYING FILES BY NAME	第03章 第02节	通过名称指定文件（通配符模式）

第五部分：本地帮助系统深度指南

5.1 man pages - 最重要的帮助工具

为什么学习man？

在实际工作中，你不能联网搜索时，man pages是你的救命稻草！每个命令的完整文档都在你的系统中。

man的诞生故事：

1971年，Dennis Ritchie在开发Unix时创建了第一版man系统。它的设计理念是"自助式文档"——每个软件自带文档，用户可以随时查阅。这种传统一直延续到今天的RHEL9。

man章节分类：

章节	名称	内容	重要性	示例命令
1	用户命令	普通用户可执行的命令	☆☆☆	ls(1), cp(1), cat(1)
2	系统调用	内核提供的函数	★	read(2), write(2)
3	库函数	标准C库函数	★	printf(3), fopen(3)
4	特殊文件	设备文件	★	tty(4), null(4)
5	文件格式	配置文件的格式	☆☆☆	passwd(5), fstab(5)
6	游戏	-	-	-
7	惯例和标准	协议、文件系统标准	★	hier(7), path_resolution(7)

章节	名称	内容	重要性	示例命令
8	系统管理	管理员命令	☆☆☆	<code>useradd(8)</code> , <code>systemctl(1)</code>

man命令基本用法：

SHELL

```
# 基本语法
man [选项] [章节] 关键词

# 查看命令帮助 (最常用)
man ls

# 查看特定章节
man 5 passwd # 查看/etc/passwd文件格式
man 8 useradd # 查看useradd管理命令

# 查看所有章节
man -a passwd

# 完整执行示例
[root@node1 ~]# man ls
LS(1)                                User Commands                                LS(1)

NAME
    ls - list directory contents

SYNOPSIS
    ls [OPTION]... [FILE]...

DESCRIPTION
    List information about the FILES (the current directory by default).
    Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.

    By default, ls does not display files whose names begin with "."
    (hidden files).

    Mandatory arguments to long options are mandatory for short options
    too.

... (按空格向下翻页)

# 输出说明：
# LS(1) = 命令名称和章节号
# NAME = 命令名称和简短描述
# SYNOPSIS = 命令语法概要
# DESCRIPTION = 详细描述
```

man页面导航快捷键（必须掌握）：

按键	功能	使用场景
空格 或 Page Down	向下翻页	浏览长文档
b 或 Page Up	向上翻页	返回查看内容
Enter	向下滚动一行	精确阅读
/关键词	搜索关键词	快速定位
n	跳转到下一个搜索结果	继续查找
N	跳转到上一个搜索结果	反向查找
gg	跳到页面开头	从头查看
G	跳到页面末尾	查看底部
q	退出man页面	完成查看

man搜索实战示例：

SHELL

```
# 示例：在ls的man页面中搜索 "-a" 选项
[root@node1 ~]# man ls
# 按 / 进入搜索模式
/-a
# 按n查找下一个包含"-a"的位置
# 按N查找上一个包含"-a"的位置
# 按q退出

# 搜索结果示例：
-a, --all
    do not ignore entries starting with .

-A, --almost-all
    do not list implied . and ..

--author
    with -l, print the author of each file
```

man页面结构详解：

找不到“assets/man-page-structure.excalidraw”。

man高级搜索技巧：

SHELL

```
# man -k 关键字搜索 (在所有man页面中搜索)
[root@node1 ~]# man -k copy
cp (1)          - copy files and directories
cpio (1)         - copy files to and from archives
scp (1)          - OpenSSH secure file copy
... (更多结果)

# man -K 全文搜索 (搜索man页面内容, 速度慢)
[root@node1 ~]# man -K "recursive search"

# whatis - 显示命令简短描述 (来自man数据库)
[root@node1 ~]# whatis ls
ls (1)          - list directory contents

[root@node1 ~]# whatis passwd
passwd (1)      - update user's authentication tokens
passwd (5)      - password file
```

🚨 技巧：

- 1. 时间管理：时间有限时，不要通读整个man页面，使用 / 搜索特定选项
- 2. 常见章节：最常用的是1（命令）、5（配置文件）、8（管理命令）
- 3. SELinux帮助：遇到SELinux问题时，man semanage-fcontext 是救命稻草

NAME	- 命令名称和简短描述
SYNOPSIS	- 命令语法概要
DESCRIPTION	- 详细描述
OPTIONS	- 选项说明
EXAMPLES	- 使用示例
FILES	- 相关文件
SEE ALSO	- 相关主题

5.2 info命令 (不考, 也不重要)

info - 更详细的帮助：

```
# 查看info文档
info ls

# 说明:
# info提供比man更详细的文档
# 使用方向键导航, q退出
```

5.3 命令自带帮助

--help 选项:

```
# 查看命令简要帮助
ls --help

# 参数说明:
# --help = 显示命令帮助信息
```

输出说明:

- 显示命令的常用选项
- 比man页面更简洁
- 适合快速查看

演示：文件管理实践

演示目标

综合运用文件管理命令完成实际任务

演示步骤

步骤1: 创建目录结构

```
# 创建项目目录结构
mkdir -p /tmp/project/{src,doc,test}

# 参数说明:
# -p = 自动创建父目录
# {} = 大括号扩展, 生成多个目录
```

步骤2: 复制文件

```
# 复制配置文件到项目目录
cp /etc/hosts /tmp/project/src/
cp /etc/hostname /tmp/project/src/

# 查看复制结果
ls -l /tmp/project/src/
```

步骤3：创建链接

```
# 创建配置文件的软链接
ln -s /tmp/project/src/hosts /tmp/project/hosts-link

# 查看链接
ls -l /tmp/project/hosts-link
```

步骤4：查找帮助

```
# 查看cp命令的帮助
man cp

# 在man页面中搜索 -r 选项
# 输入 /-r 然后按 n 查找下一个
```

练习：上午练习

练习目标

掌握文件管理、链接创建和帮助系统使用

练习任务

任务1：文件操作练习

```
# 1. 在/tmp下创建exercise目录
mkdir /tmp/exercise

# 2. 复制/etc/passwd到exercise目录
cp /etc/passwd /tmp/exercise/

# 3. 重命名为users.txt
mv /tmp/exercise/passwd /tmp/exercise/users.txt
```

任务2：链接练习

```
# 1. 创建users.txt的硬链接
ln /tmp/exercise/users.txt /tmp/exercise/users-hard

# 2. 创建users.txt的软链接
ln -s /tmp/exercise/users.txt /tmp/exercise/users-soft

# 3. 查看两个链接的inode
ls -il /tmp/exercise/users*
```

任务3：帮助系统练习

```
# 1. 查看ls命令的man页面
man ls

# 2. 搜索 -a 选项
# 按 /-a Enter, 然后按 n 查找

# 3. 查看mkdir的帮助
mkdir --help
```

相关练习：RH124V9 教案练习

练习名称	对应章节	说明
总练习：GETTING HELP IN RED HAT ENTERPRISE LINUX	第04章	通过本地帮助系统获取信息

下午 - 文本编辑与用户管理

第六部分：重定向和管道

课前故事：Unix管道的诞生

1973年，Doug McIlroy在贝尔实验室编写Fortran程序时，遇到了一个重复的问题：如何让多个程序协同工作？

当时的做法是：

- 将第一个程序的输出保存到临时文件
- 第二个程序读取这个临时文件
- 处理完再删除临时文件

这种方法既繁琐又浪费磁盘空间。McIlroy提出了一个革命性的想法：为什么不能让一个程序的输出直接成为另一个程序的输入？

于是，管道(pipe) 诞生了！

这个简洁的  符号，成为了Unix哲学的体现："做好一件事，并协同工作"。

6.1 标准输入输出

三个标准文件描述符：

描述符	名称	默认位置	用途	符号表示
0	标准输入(stdin)	键盘	程序读取数据	<
1	标准输出(stdout)	屏幕	正确输出	> >>
2	标准错误(stderr)	屏幕	错误输出	2> 2>>

为什么需要了解这三个通道？

在Linux系统中，一切皆文件——包括你的键盘输入和屏幕输出！每个程序启动时，系统会自动打开这三个"文件"：

Standard I/O File Descriptors

找不到"assets/day02-stdio.excalidraw"。

Three Standard Channels:

- **Channel 0 (stdin)** → Standard Input → From keyboard
- **Channel 1 (stdout)** → Standard Output → To screen
- **Channel 2 (stderr)** → Standard Error → To screen

⚠ 易错点：

- 标准输出和标准错误默认都显示在屏幕上，但它们是独立的通道
- 重定向时，> 只重定向标准输出，不会重定向标准错误
- 要同时重定向，必须使用 &> 或分别重定向 > 和 2>

6.2 输出重定向

重定向标准输出：

SHELL

```
# 将输出保存到文件（覆盖模式）
ls -l /etc > /tmp/ls-output.txt

# 参数说明：
# > = 重定向标准输出到文件，覆盖原有内容

# ⚠ 警告：如果文件已存在，内容会被完全覆盖！
```

输出示例：

```
[root@node1 ~]# ls -l /etc | head -n 3
total 1400
drwxr-xr-x. 135 root root 8192 Jan 15 10:00 .
dr-xr-xr-x. 18 root root 244 Jan 15 09:00 ..
-rw-r--r--. 1 root root 1024 Jan 10 00:00 adjtime

# 查看保存的内容
[root@node1 ~]# cat /tmp/ls-output.txt | head -n 3
total 1400
drwxr-xr-x. 135 root root 8192 Jan 15 10:00 .
drwxr-xr-x. 18 root root 244 Jan 15 09:00 ..
```

Redirection Comparison

找不到“assets/day02-redirection.excalidraw”。

Redirection Operators:

- `>` or `1>` → Redirect stdout (overwrite mode)
 - `>>` or `1>>` → Redirect stdout (append mode)
 - `2>` → Redirect stderr (overwrite mode)
 - `2>>` → Redirect stderr (append mode)
 - `&>` → Redirect both stdout and stderr (overwrite)
 - `&>>` → Redirect both stdout and stderr (append)
- 默认标准输出和标准错误输出都输出到屏幕, 除非使用重定向操作符:
- `>` - 将标准输出覆盖写入文件
 - `>>` - 将标准输出追加到文件末尾
 - `2>` - 只重定向标准错误
 - `&>` - 重定向所有输出

追加到文件:

```
# 将输出追加到文件末尾 (不覆盖)
date >> /tmp/ls-output.txt

# 参数说明:
# >> = 追加重定向, 在文件末尾添加新内容

# 输出示例:
[root@node1 ~]# date >> /tmp/ls-output.txt
[root@node1 ~]# tail -n 1 /tmp/ls-output.txt
Wed Jan 15 10:30:00 CST 2025
```

⚠ 场景辨析:

- `>` = 覆盖模式 (清空原文件, 写入新内容)
- `>>` = 追加模式 (保留原内容, 在末尾添加)

常见任务:

```
# 场景: 将/etc/hosts的内容追加到/tmp/hosts-backup
cat /etc/hosts >> /tmp/hosts-backup

# 场景: 将当前时间保存到/tmp/timestamp (新文件)
date > /tmp/timestamp
```

重定向标准错误:

```
# 将错误信息重定向到文件
find /etc -name passwd 2> /tmp/errors.txt

# 参数说明:
# 2> = 重定向标准错误(channel 2)
# 注意: 数字2前面没有空格!

# 完整输出示例:
[root@node1 ~]# find /etc -name passwd
/etc/passwd                               ← 正确输出 (stdout)
find: '/etc/audit': Permission denied ← 错误输出 (stderr)
find: '/etc/sssdd': Permission denied ← 错误输出 (stderr)

# 只重定向错误:
[root@node1 ~]# find /etc -name passwd 2> /tmp/errors.txt
/etc/passwd                               ← 只显示正确输出

# 查看错误信息:
[root@node1 ~]# cat /tmp/errors.txt
find: '/etc/audit': Permission denied
find: '/etc/sssdd': Permission denied
```

⚠ 注意: 文件描述符数字和 > 之间不能有空格!

-  正确: 2> errors.txt
-  错误: 2 > errors.txt

```
# 方法1: 使用 &> (推荐, 常用)
ls /etc /nonexistent &> /tmp/all-output.txt

# 参数说明:
# &> = 同时重定向stdout和stderr到同一文件

# 方法2: 分别重定向
ls /etc /nonexistent > /tmp/output.txt 2> /tmp/errors.txt

# 方法3: 先重定向stdout, stderr跟随stdout
ls /etc /nonexistent > /tmp/all-output.txt 2>&1
```

重定向对比表 (工作常用) :

符号	名称	重定向内容	覆盖/追加
>	输出重定向	标准输出(1)	覆盖
>>	追加重定向	标准输出(1)	追加
2>	错误重定向	标准错误(2)	覆盖
2>>	错误追加重定向	标准错误(2)	追加
&>	全部重定向	stdout+stderr	覆盖
&>>	全部追加重定向	stdout+stderr	追加

重定向到/dev/null:

```
# 丢弃错误信息(避免错误信息显示在屏幕)
find /etc -name passwd 2> /dev/null

# 说明:
# /dev/null 是"黑洞"设备, 写入的数据会永久丢失
# 用于丢弃不需要的输出, 保持终端整洁

# 示例: 只显示找到的文件, 不显示权限错误
[root@node1 ~]# find / -name "hosts" 2> /dev/null
/etc/hosts
~/.ssh/known_hosts

# 不使用 /dev/null 的情况:
[root@node1 ~]# find / -name "hosts"
find: '/root/.cache': Permission denied
find: '/proc/tty/driver': Permission denied
/etc/hosts                                     ← 结果夹杂在错误中
find: '/var/log/audit': Permission denied
```

⚠ 技巧:

- 遇到"搜索并显示结果"题目时, 使用 `2> /dev/null` 避免错误干扰
- 题目要求"静默执行"时, 使用 `> /dev/null 2>&1` 或 `&> /dev/null`

6.3 管道

什么是管道?

管道  是Unix/Linux最强大的特性之一, 它将前一个命令的标准输出(stdout)直接连接到后一个命令的标准输入(stdin)。

管道工作原理:

Traditional vs Pipe Method

找不到"assets/day02-pipe-comparison.excalidraw"。

How Pipe Works:

找不到"assets/day02-pipe-flow.excalidraw"。

⚠ 管道的重要特性:

- 只连接标准输出(stdout), 不包括标准错误(stderr)
- 数据流是单向的, 从左到右
- 可以链式连接多个命令: `cmd1 | cmd2 | cmd3 | cmd4`

管道示例1: 分页查看 (常用)


```
# 分页查看长输出
ls -l /usr/bin/ | less

# 参数说明:
# | = 管道符, 将ls的输出传递给less
# less = 分页查看工具

# 在less中的操作:
# 空格键 = 向下翻页
# b键 = 向上翻页
# q键 = 退出
# /关键字 = 向下搜索
# ?关键字 = 向上搜索

# 输出示例:
total 368536
-rwxr-xr-x. 1 root root    67856 Jan 10 00:00 [
-rwxr-xr-x. 1 root root   115400 Jan 10 00:00 a2p
-rwxr-xr-x. 1 root root   529800 Jan 10 00:00 a2ps
... (数万行输出, 使用less可以方便浏览)
```

⚠ 场景:

- 题目要求"查看某目录的文件列表"时, 如果目录内容很多, 记得用 `| less` 分页查看
- 例如: `ls -l /etc | less` 查看/etc目录的所有内容

管道示例2: 统计行数 (重要)

```
# 统计文件/目录数量
ls | wc -l

# 参数说明:
# ls = 列出当前目录内容
# | = 管道符
# wc = word count, 统计命令
# -l = --lines, 统计行数

# 输出示例:
[root@node1 ~]# ls /etc
adjtime      aliases      cron.d       fstab        hosts        nsswitch.conf passwd
audit        alternatives  cron.daily   group        hostname     passwd-      profile
... (更多文件)

[root@node1 ~]# ls /etc | wc -l
185
# 说明: /etc目录有185个文件和目录
```

⚠ 技巧:

- `wc -l` = 统计行数
- `wc -w` = 统计单词数
- `wc -c` = 统计字符数
- `wc` = 同时显示行数、单词数、字符数

常见任务:

```
# 场景: 统计/etc目录下有多少个conf文件
ls /etc/*.conf 2>/dev/null | wc -l

# 场景: 统计/etc/passwd文件有多少行 (即多少用户)
wc -l /etc/passwd
```

管道示例3：查看前/后N行

SHELL

```
# 查看最新的10个文件（按修改时间排序）
ls -t /var/log | head -n 10

# 参数说明：
# ls -t = 按修改时间排序（最新的在前）
# head -n 10 = 显示前10行

# 输出示例：
[root@node1 ~]# ls -t /var/log | head -n 5
lastlog          ← 最新
wtmp
btmp
messages
maillog

# 查看最大的5个文件
ls -S /var/log | head -n 5

# 参数说明：
# ls -S = 按文件大小排序（最大的在前）
```

head和tail对比：

命令	功能	常用选项
head	显示文件开头	-n 10 前10行
tail	显示文件末尾	-n 10 后10行
tail -f	实时跟踪文件增长	-f follow模式

⚠ 重点：tail -f 日志查看

SHELL

```
# 实时查看系统日志
tail -f /var/log/messages

# 说明：
# -f = --follow, 实时跟踪日志增长
# 按Ctrl+C退出

# 常见场景：查看服务启动日志
systemctl start httpd
journalctl -u httpd -f ← 使用journalctl实时查看服务日志
```

管道示例4：搜索内容（重要）

```
# 搜索包含ssh的进程
ps aux | grep ssh

# 参数说明:
# ps aux = 查看所有进程详细信息
# grep ssh = 搜索包含"ssh"的行

# 输出示例:
[root@node1 ~]# ps aux | grep ssh
root      12345  0.0  0.1 11200 4500 ?        Ss   10:00   0:00 /usr/sbin/sshd -D
root      23456  0.0  0.2 15400 6700 ?        S    10:05   0:00 sshd: student@pts/0
student   23567  0.0  0.1 12100 3400 pts/0    S+   10:10   0:00 grep --color=auto ssh
```

注意：这行是grep进程本身

⚠ grep技巧:

- 使用 `grep -v` 排除内容
- 使用 `grep -i` 忽略大小写
- 使用 `grep -r` 递归搜索目录
- 使用 `grep --color` 高亮显示（默认已启用）

grep常见用法（必记）：

```
# 搜索文件内容
grep "root" /etc/passwd

# 忽略大小写搜索
grep -i "ssh" /etc/services

# 递归搜索目录
grep -r "192.168.1" /etc/

# 反向匹配（排除）
grep -v "^#" /etc/httpd/conf/httpd.conf
      ↑
      排除注释行

# 显示行号
grep -n "error" /var/log/messages

# 统计匹配行数
grep -c "failed" /var/log/secure
```

管道示例5：组合使用（高级技巧）

```
# 统计ssh进程数量
ps aux | grep ssh | grep -v grep | wc -l

# 参数说明:
# ps aux | grep ssh      -> 查找包含ssh的进程
# grep -v grep           -> 排除grep进程本身
# wc -l                  -> 统计行数

# 输出示例:
[root@node1 ~]# ps aux | grep ssh | grep -v grep | wc -l
2
# 说明: 有2个ssh相关进程 (通常是sshd守护进程和当前连接)

# 查看系统登录失败的用户 (安全审计)
grep "Failed password" /var/log/secure | awk '{print $11}' | sort | uniq

# 参数说明:
# grep "Failed password" -> 找出登录失败的行
# awk '{print $11}'      -> 提取第11列 (用户名)
# sort                   -> 排序
# uniq                   -> 去重 (需要先排序)

# 输出示例:
root
admin
test
# 说明: root、admin、test用户尝试登录失败过
```

⚠ 管道使用原则:

- 1. 从左到右理解: 第一个命令的输出是第二个命令的输入
- 2. 先测试再连接: 单独运行每个命令, 确保正常后再用管道连接
- 3. 错误不传递: 管道只传递标准输出, 不传递标准错误
- 4. 合理使用: 2-3个命令的管道是合适的, 过长会难以理解

管道 vs 重定向 (重要区分):

特性	管道 ` `	重定向 `>`
连接对象	命令到命令	命令到文件
数据方向	单向传递	覆盖/追加
是否创建文件	否	是
典型用途	命令组合	保存结果

使用场景示例:

```
# 使用管道: 查看并过滤
ls /etc | grep conf

# 使用重定向: 保存结果
ls /etc > /tmp/file-list.txt

# 组合使用: 过滤并保存
ls /etc | grep conf > /tmp/conf-files.txt
```

tee - 三通管道:

```
# 同时显示和保存输出
ls | tee /tmp/ls-output.txt | less

# 参数说明：
# tee = 将输出同时发送到文件和屏幕
# 先保存到文件，再传递给less
```

说明：

- 适合需要保存输出又要查看的场景
- 常用于日志记录

相关练习

练习名称	对应章节	说明
选择练习：REDIRECTING OUTPUT TO A FILE OR PROGRAM	第03章 第05节	重定向输出到文件或程序

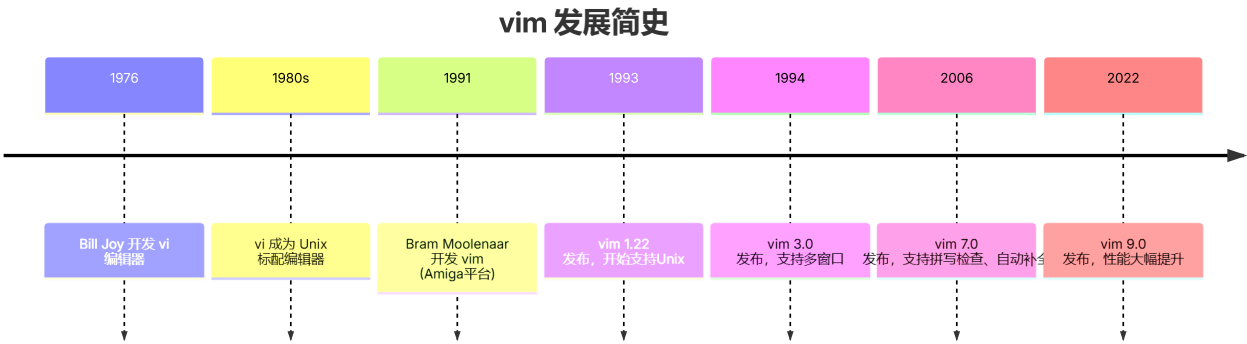
第七部分：vim编辑器

课前故事：vim的传奇诞生

1991年，一位名叫 **Bram Moolenaar** 的荷兰程序员，在买了一台**Amiga**计算机后，发现了一个问题：这台计算机上没有他习惯使用的 **vi**编辑器。

于是，他决定自己写一个！

Bram以**Stevie**（一款vi克隆）为基础，开发了一个新的编辑器。他将其命名为 **Vi IMproved**（改进的vi），简称 **vim**。



vim的独特魅力：

vim的设计理念是"不离开键盘"——所有操作都可以通过键盘完成，无需鼠标。这让熟练用户能够达到惊人的编辑速度。

⚠️ 重要性：

- vim是**RHCSA**必备工具（修改配置文件、创建脚本）
- 实际环境没有图形界面时，必须用vim
- vim基本操作是及格线技能——必须掌握

7.1 vim简介

为什么学习vim?

场景	vim优势	其他编辑器问题
远程SSH连接	文本界面，完美支持	nano功能弱，图形编辑器不可用
服务器管理	默认安装，无需额外安装	需要网络下载安装
配置文件编辑	高效快捷，支持语法高亮	不支持系统配置文件语法
脚本编写	强大的批量编辑能力	其他编辑器效率低
系统救援	最小系统也可用	GUI编辑器无法运行

⚠ 实际场景：

- 修改 `/etc/ssh/sshd_config`
- 编辑 `/etc/fstab` 添加挂载
- 配置 `/etc/systemd/system/custom.service`
- 以上都必须使用vim!

vim vs vi:

SHELL

```
# 检查系统中vi和vim的关系
[root@node1 ~]# which vi
/usr/bin/vi

[root@node1 ~]# which vim
/usr/bin/vim

# 查看别名设置
[root@node1 ~]# alias vi
alias vi='vim'

# 说明:
# RHEL9系统中，vi命令实际是vim的别名
# 两者是同一个程序
```

RHEL9的vim版本特性：

- 默认启用语法高亮
- 支持自动缩进（编辑代码时）
- 支持搜索高亮
- 支持无限撤销

⚠ 提示：

- 直接使用 `vim filename` 即可
- 如果提示命令不存在，使用 `vi filename`（实际也是vim）
- 不要尝试安装其他编辑器，浪费时间

7.2 vim操作模式

vim的核心：模式化编辑

vim与其他编辑器最大的不同是：它有多个模式，不同模式下按键的含义不同！

四种模式详解：

找不到“assets/day02-vim-modes.excalidraw”。

模式详细说明：

模式	名称	进入方式	退出方式	用途	重要性
Command	命令模式/普通模式	默认启动	-	移动光标、复制、粘贴、删除	☆☆☆
Edit	编辑模式/插入模式	按 i 或 a	按 Esc	输入文字内容	☆☆☆
Extended	底行模式/末行模式	按 :	按 Esc	保存和退出	☆☆☆
Visual	可视模式	按 v 或 V	按 Esc	选择文本块（高级）	☆

⚠️ 最常见错误：

- ✗ 打开文件后直接输入 → 屏幕什么都不显示（因为你在Command模式！）
- ✓ 正确做法：打开文件 → 按 **i** 进入Edit模式 → 开始输入

模式快速切换记忆口诀：

i → 进入编辑 (insert)
Esc → 返回命令 (escape)
: → 保存退出 (colon)

模式状态提示（如何知道自己在哪个模式？）：

模式	屏幕底部显示	光标状态
Command	无显示或显示文件信息	闪烁的方块
Edit	显示 -- INSERT -- 或 -- 插入 --	闪烁的竖线
Extended	底部出现 : 冒号	不显示
Visual	显示 -- VISUAL -- 或 -- 可视 --	高亮选中区域

⚠️ 技巧：

- 如果不知道当前在哪个模式，连续按几次**Esc**键，确保回到Command模式
- 看到 **-- INSERT --** 才能开始输入文字
- 看到 **:** 才能输入 **:wq** 保存退出

7.3 基本操作

打开/创建文件：

```
# 打开或创建文件
vim /tmp/test.txt

# 参数说明：
# /tmp/test.txt = 文件路径
# 文件不存在 → 创建新文件
# 文件存在    → 打开现有文件

# 同时打开多个文件
vim file1.txt file2.txt file3.txt
# 在vim中切换：:n 下一个，:N 上一个

# 打开文件并跳到第10行
vim +10 /tmp/test.txt

# 以只读模式打开
vim -r /tmp/test.txt
```

⚠ 场景：

```
# 场景：在/root目录创建config.txt文件并配置
vim /root/config.txt

# 操作流程：
# 1. 文件不存在，vim会自动创建
# 2. 按i进入编辑模式
# 3. 输入配置内容
# 4. 按Esc返回命令模式
# 5. 输入:wq保存退出
```

保存和退出（必须掌握）：

在**Command**模式下（按  进入Extended模式）：

命令	功能	使用场景	重要性
:w	保存，继续编辑	编辑中定期保存	★★
:wq	保存并退出	完成编辑	★★★
:q	不保存退出	未修改时退出	★
:q!	强制退出，放弃修改	编辑错了要重来	★★★★
:x	保存并退出（同:wq）	完成编辑	★★
ZZ	保存并退出（Command模式）	快捷退出	★★

⚠ 必记：保存退出的3种方法

```
# 方法1：最常用（推荐）
:wq

# 方法2：等效
:x

# 方法3：Command模式下（不按:）
# ⚠ 注意：如果编辑错了，不要保存！
ZZ

# 强制退出，放弃所有修改，重新来过
:q!
```


常用编辑命令（Command模式）：

插入命令（从Command进入Edit模式）：

按键	功能	使用场景
i	在光标前插入	最常用，默认选择
a	在光标后插入	想在光标后继续输入
I	在行首插入	需要在行首添加内容
A	在行尾插入	需要在行尾追加内容
o	在下方新建一行插入	想在当前行后新增内容
O	在上方新建一行插入	想在当前行前新增内容

⚠️ 场景辨析：

SHELL

```
# 场景1：编辑配置文件，在文件末尾添加配置
# 操作：按 G（跳到文件末尾）→ 按 o（新建一行）→ 输入配置

# 场景2：在文件开头添加注释
# 操作：按 gg（跳到文件开头）→ 按 I（行首插入）→ 输入 #

# 场景3：修改当前行的某个字
# 操作：移动光标到目标位置 → 按 i → 修改
```

删除命令（Command模式）：

按键	功能	示例	重要性
x	删除光标处字符	删除一个字	★★
X	删除光标前字符	删除前一个字	★
dd	删除当前行	删除一行	★★★★
dw	删除到单词末尾	删除一个单词	★★
d\$	删除到行尾	删除从光标到行尾	★★
d0	删除到行首	删除从光标到行首	★★
3dd	删除3行	删除多行	★★★★
dG	删除到文件末尾	清空后半部分	★
dgg	删除到文件开头	清空前半部分	★

⚠️ 实际场景：

SHELL

```
# 场景：删除/etc/hosts文件的第5行
vim /etc/hosts
# 操作：移动到第5行（:5 或 5G）→ 按 dd

# 场景：删除文件的所有内容
vim /tmp/file.txt
# 操作：gg（跳到开头）→ dG（删除到末尾）

# 场景：删除当前行光标后的所有内容
# 操作：d$
```

复制和粘贴（Command模式）：

按键	功能	说明
yy	复制当前行	yank（复制）
3yy	复制3行	复制多行
p	在光标后粘贴	paste（粘贴）
P	在光标前粘贴	在前面粘贴
dd	删除当前行	实际是剪切（cut）

⚠️ 技巧：

SHELL

场景：复制配置文件中的一行，然后粘贴多行
vim /etc/hosts
操作：
1. 移动到目标行
2. yy（复制）
3. p（粘贴）、p（粘贴）、p（粘贴）→ 粘贴3次

场景：移动一行到另一个位置
操作：
1. dd（删除=剪切）
2. 移动到目标位置
3. p（粘贴）

撤销和重做（救命命令）：

按键	功能	说明	重要性
u	撤销上一次操作	undo	★★★★
Ctrl+r	重做（撤销撤销）	redo	★★★
U	撤销对当前行的所有修改	大写U	★

⚠️ 实际场景：

SHELL

场景：编辑错了，不要慌！
操作：按 u 撤销，可以多次按u，一直撤销到满意为止

场景：撤销多了，要恢复
操作：按 Ctrl+r 恢复

建议：每隔几秒钟按一次u确认可以撤销

光标移动（Command模式）：

基础移动：

按键	功能	记忆技巧	重要性
h j k l	左下上右	hjkl在键盘上形成方向	★★★

按键	功能	记忆技巧	重要性
   	方向键有时也可以用	更直观	☆☆☆

快速移动：

按键	功能	使用场景	重要性
	跳到下一个单词开头	word	☆☆
	跳到上一个单词开头	back	☆☆
	跳到当前单词末尾	end	☆
	跳到行首	行的开头是0	☆☆☆
	跳到行尾	正则表达式\$表示行尾	☆☆☆
	跳到行首第一个非空字符	忽略行首空格	☆

文件内移动：

按键	功能	使用场景	重要性
	跳到文件第1行	go to start	☆☆☆
	跳到文件最后一行	Go to end	☆☆☆
	跳到第10行	跳到指定行号	☆☆☆
	跳到文件50%位置	跳到文件中间	☆

屏幕移动：

按键	功能	说明
	向下翻页	forward
	向上翻页	backward
	向下翻半屏	down
	向上翻半屏	up
	跳到屏幕顶部/中部/底部	High Middle Low

⚠ 必记组合：

SHELL

```
# 快速跳到指定行号（题目常要求）
vim +20 /etc/passwd      → 打开文件直接到第20行
vim /etc/passwd +20      → 另一种写法
# 在vim中：:20 或 20G    → 跳到第20行

# 快速跳到文件首尾
gg      → 文件开头（查看配置文件开头说明）
G      → 文件末尾（在末尾添加配置）
```

搜索和替换（高级技巧）：

搜索（Command模式）：

按键	功能	说明
/text	向下搜索text	按n找下一个，N找上一个
?text	向上搜索text	按n找上一个，N找下一个
n	跳到下一个搜索结果	next
N	跳到上一个搜索结果	previous

替换（Extended模式）：

命令	功能	示例
:s/old/new/	替换当前行第一个old为新	:s/root/admin/
:s/old/new/g	替换当前行所有old为新	:s/root/admin/g
:%s/old/new/g	替换全文所有old为新	:%s/root/admin/g
:%s/old/new/gc	替换全文所有old为新（需确认）	逐个确认

⚠ 场景：

SHELL

```
# 场景：将文件中所有的"eth0"替换为"ens3"
vim /etc/sysconfig/network-scripts/ifcfg-eth0
# 操作：
# 1. 打开文件后按 : 进入Extended模式
# 2. 输入： %s/eth0/ens3/g
# 3. 按Enter执行，但不要保存

# 场景：查看文件中包含"SELinux"的行
vim /etc/selinux/config
# 操作：
# 1. 打开文件后按 /
# 2. 输入： SELinux
# 3. 按Enter，按n查找下一个
```

演示：vim实践

演示目标

掌握vim基本操作

演示步骤

步骤1：创建新文件

SHELL

```
# 使用vim创建新文件
vim /tmp/hello.txt

# 进入vim后，按 i 进入编辑模式
```

步骤2：输入内容

在编辑模式下输入：

```
Hello, RHCSA!  
This is my first vim file.  
Learning Linux is fun!
```

步骤3：保存退出

```
# 按 Esc 返回Command模式  
# 输入 :wq 保存退出  
:wq
```

步骤4：验证内容

```
# 查看文件内容  
cat /tmp/hello.txt
```

练习：vim练习

练习目标

熟练使用vim编辑文本文件

练习任务

任务1：创建配置文件

```
# 使用vim创建配置文件  
vim /tmp/app.conf  
  
# 输入以下内容：  
# Application Configuration  
version=1.0  
debug=true  
logfile=/var/log/app.log
```

任务2：修改文件

```
# 重新打开文件
vim /tmp/app.conf

# 练习:
# 1. 将debug=true改为debug=false
# 2. 在文件末尾添加新行: author=student
# 3. 保存退出
```

任务3：学习vimtutor

```
# 运行vim交互式教程
vimtutor

# 说明:
# 这是vim官方提供的教程
# 大约需要1小时完成
# 按照教程提示操作即可
```

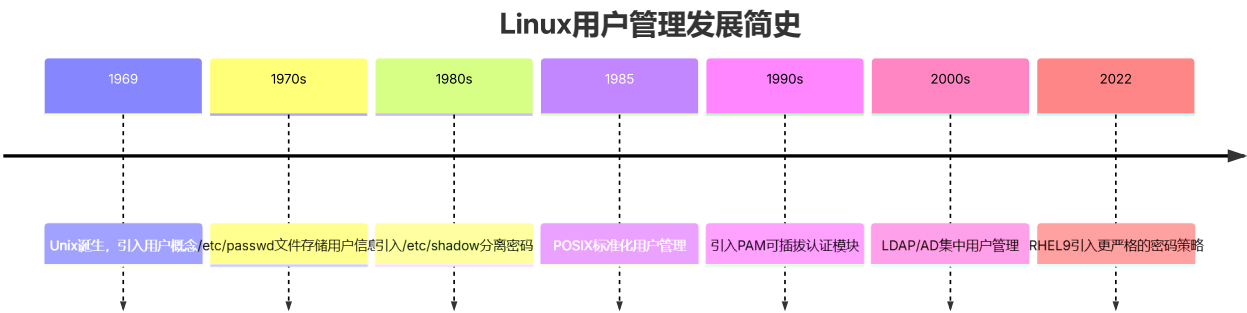
练习名称	对应章节	说明
练习：从 SHELL 提示符编辑文本文件	第05章 第02节	vim编辑器基础操作
总练习：文本文件的创建、查找和编辑	第05章	文本编辑综合练习

第八部分：用户和组管理

课前故事：Linux用户管理的诞生

1969年，当Ken Thompson和Dennis Ritchie在贝尔实验室开发Unix时，他们面临一个挑战：如何让多个用户安全地共享同一台大型计算机？

当时的解决方案是：用户身份和权限隔离。



Linux用户管理的核心理念：

Multi-User System Architecture

找不到“assets/day02-multiuser-system.excalidraw”。

Each User Properties:

- Unique user ID (UID)
- Separate home directory (e.g., /home/user)
- Independent permission settings
- Can belong to multiple groups (primary + supplementary)

⚠️ 重要性:

- 用户和组管理是RHCSA重要内容
- 实际工作中常要求创建特定用户、设置组权限
- 理解UID/GID对权限管理至关重要

8.1 用户概念

Linux用户类型（必记）：

类型	名称	UID范围	能否登录	用途	重要性
超级用户	root	0	是	系统管理，最高权限	★★★★
系统用户	系统服务	1-999	否	运行系统服务（apache、nginx等）	★★★
普通用户	日常用户	1000-60000	是	日常工作和操作	★★★★

重要说明:

- **UID 0** = root用户，无论用户名是什么，UID为0就是root
- **UID 1-200** = RHEL9预留的系统用户
- **UID 201-999** = 用户安装的系统服务
- **UID 1000+** = 普通用户，第一个普通用户默认UID为1000

⚠️ 注意

- 不要修改UID为0的用户，除非你完全理解后果
- 系统用户不要设置为可登录（安全风险）
- RHEL7之前UID从500开始，RHEL7+从1000开始

用户信息存储文件:

文件	作用	权限	重要性
/etc/passwd	用户账户信息	所有用户可读	★★★★
/etc/shadow	用户密码哈希	仅root可读	★★★
/etc/group	组信息	所有用户可读	★★★★
/etc/gshadow	组密码信息	仅root可读	★

/etc/passwd文件格式:

```
root:x:0:0:root:/root:/bin/bash
```



字段说明:

字段1: 登录名 (登录时输入的用户名)

字段2: 密码占位符 (历史遗留, 现在都是x)

字段3: UID (用户ID, 0=root, 1-999=系统, 1000+=普通)

字段4: GID (主组ID, 对应/etc/group中的GID)

字段5: 注释信息 (可存放全名、电话、办公室等)

字段6: 家目录 (用户登录后的默认目录)

字段7: 登录Shell (/bin/bash可登录, /sbin/nologin不可登录)

🚨 场景:

查看/etc/passwd了解用户信息

```
cat /etc/passwd
```

输出示例:

```
root:x:0:0:root:/root:/bin/bash
```

```
bin:x:1:1:bin:/bin:/sbin/nologin
```

```
daemon:x:2:2:daemon:/sbin:/sbin/nologin
```

```
adm:x:3:4:adm:/var/adm:/sbin/nologin
```

```
student:x:1000:1000:Student User:/home/student:/bin/bash
```

分析:

- root UID=0, 可以登录

- bin, daemon, adm UID<1000, 系统用户, 不能登录(/sbin/nologin)

- student UID=1000, 普通用户, 可以登录(/bin/bash)

查看用户信息:


```
# 查看当前用户信息
id

# 输出示例:
uid=1000(student) gid=1000(student) groups=1000(student),10(wheel)

# 输出说明:
# uid=1000(student)  → 用户ID和用户名
# gid=1000(student)  → 主组ID和组名
# groups=            → 所属的所有组
#   1000(student)    → 主组
#   10(wheel)        → 附加组

# 查看指定用户信息
id root
uid=0(root) gid=0(root) groups=0(root)

# 查看当前登录用户名
whoami
student

# 查看所有登录用户
who
student pts/0 2025-01-15 10:00 (192.168.1.10)

# 查看用户详细信息 (从passwd读取)
getent passwd student
student:x:1000:1000:Student User:/home/student:/bin/bash
```

- 🚨 技巧:
- `id` 显示最完整的用户信息 (UID、GID、所属组)
 - `whoami` 只显示用户名, 快速确认身份
 - `who` 显示所有登录用户, 判断是否有其他用户在线

8.2 用户管理

useradd - 创建用户 (必考命令) :

基本语法:

```
useradd [选项] 用户名
```

常用选项 (必记) :

选项	长选项	功能	示例	重要性
-u	--uid	指定UID	<code>useradd -u 2000 test</code>	★★
-g	--gid	指定主组	<code>useradd -g 1000 test</code>	★★★
-G	--groups	指定附加组	<code>useradd -G wheel test</code>	★★★
-d	--home-dir	指定家目录	<code>useradd -d /data/test test</code>	★★
-m	--create-home	创建家目录	<code>useradd -m test</code>	★★★
-M	--no-create-home	不创建家目录	<code>useradd -M test</code>	★
-s	--shell	指定登录Shell	<code>useradd -s /bin/bash test</code>	★★
-c	--comment	添加注释	<code>useradd -c "Admin User" admin</code>	★

创建用户示例:

SHELL

```
# 示例1: 基本创建 (最常用)
useradd tom

# 说明:
# 自动分配UID (从1000开始)
# 自动创建同名组 (作为主组)
# 自动创建家目录 (/home/tom)
# 默认Shell为/bin/bash

# 验证创建结果

[root@node1 ~]# id tom
uid=1001(tom) gid=1001(tom) groups=1001(tom)

[root@node1 ~]# ls -ld /home/tom
drwx-----. 2 tom tom 62 Jan 15 10:30 /home/tom
```

↑ ↑

| └─ 所有组 (主组)

└── 所有者

SHELL

```
# 示例2: 创建用户并指定附加组 (常见)
useradd -G wheel alice

# 参数说明:
# -G wheel → 将alice添加到wheel组 (sudo组)
# wheel组成员可以使用sudo

# 验证组配置
[root@node1 ~]# id alice
uid=1002(alice) gid=1002(alice) groups=1002(alice),10(wheel)
```

↑
附加组

SHELL

```
# 示例3：创建系统用户
useradd -r -s /sbin/nologin -M nginx

# 参数说明：
# -r → --system, 创建系统用户 (UID<1000)
# -s → 指定shell为/sbin/nologin (不可登录)
# -M → 不创建家目录

# 验证
[root@node1 ~]# id nginx
uid=990(nginx) gid=985(nginx) groups=985(nginx)
```

↑
系统用户UID<1000

⚠️ 常见任务:

SHELL

```
# 场景：创建用户admin，并加入wheel组
useradd -G wheel admin

# 场景：创建用户bob，UID为2000
useradd -u 2000 bob

# 场景：创建系统用户myservice，不可登录
useradd -r -s /sbin/nologin myservice
```

usermod - 修改用户（必考命令）：

基本语法:

常用选项：

选项	长选项	功能	示例	重要性
-aG	--append --groups	追加组（重要）	usermod -aG wheel tom	☆☆☆
-g	--gid	修改主组	usermod -g 1001 tom	☆☆
-l	--login	修改登录名	usermod -l thomas tom	☆☆
-d	--home-dir	修改家目录	usermod -d /data/tom tom	☆
-m	--move-home	移动家目录内容	usermod -md /data/tom tom	☆
-s	--shell	修改登录Shell	usermod -s /bin/zsh tom	☆☆
-L	--lock	锁定用户	usermod -L tom	☆☆
-U	--unlock	解锁用户	usermod -U tom	☆☆

⚠ 必记：-aG vs -G

```
# X 错误: usermod -G wheel tom
# 结果: tom只会属于wheel组, 原来的组全部丢失!

# ✔ 正确: usermod -aG wheel tom
# 结果: tom在原有组的基础上, 追加wheel组
# -a = append (追加), 必须和-G一起使用

# 示例对比:
[root@node1 ~]# id tom
uid=1001(tom) gid=1001(tom) groups=1001(tom)

# 使用-G (覆盖) :
[root@node1 ~]# usermod -G wheel tom
[root@node1 ~]# id tom
uid=1001(tom) gid=1001(tom) groups=10(wheel)
# 原来的tom组不见了!

# 使用-aG (追加) :
[root@node1 ~]# usermod -aG tom tom
[root@node1 ~]# id tom
uid=1001(tom) gid=1001(tom) groups=1001(tom),10(wheel)
# 原有的组保留了, 新增了wheel组
```

修改用户示例：

```
# 示例1: 添加用户到多个组
usermod -aG wheel,developers,sysadmin tom

# 注意: 多个组用逗号分隔, 不能有空格
# 参数说明:
# -aG wheel,developers,sysadmin → 同时添加到多个组

# 验证
[root@node1 ~]# groups tom
tom : tom wheel developers sysadmin
```

```
# 示例2: 修改用户登录名
useimod -l thomas tom

# 说明:
# 修改登录名从tom改为thomas
# 家目录不会自动改名
# UID/GID保持不变

# 验证
[root@node1 ~]# id thomas
uid=1001(thomas) gid=1001(tom) groups=1001(tom)
# UID没变, 但用户名改了
```

⚠ 实际场景:

```
# 场景: 将用户alice添加到wheel组
useimod -aG wheel alice

# 场景: 将用户bob重命名为robert
useimod -l robert bob
```

userdel - 删除用户:

基本语法:

```
userdel [选项] 用户名
```

选项:

选项	长选项	功能	重要性
无	-	删除用户, 保留家目录	★★
-r	--remove	删除用户及其家目录	★★★

删除用户示例:

```
# 示例1: 删除用户但保留家目录
userdel tom

# 说明:
# 删除用户账户
# /etc/passwd中的记录被删除
# /home/tom 家目录保留
# 可以用于备份或数据分析

# 验证
[root@node1 ~]# id tom
id: 'tom': no such user
# 用户不存在

[root@node1 ~]# ls -ld /home/tom
drwx-----. 2 1001 1001 62 Jan 15 10:30 /home/tom
# 家目录还在, 但所有者显示为UID
```

```
# 示例2: 完全删除用户 (常用)
userdel -r alice

# 说明:
# 删除用户账户
# 删除家目录 /home/alice
# 删除邮件池 /var/spool/mail/alice

# ⚠️ 警告: -r选项不可逆, 删除前确保没有重要数据!

# 验证
[root@node1 ~]# id alice
id: 'alice': no such user

[root@node1 ~]# ls -ld /home/alice
ls: cannot access '/home/alice': No such file or directory
# 家目录也被删除了
```

⚠️ 注意:

- 删除用户前, 确认是否需要保留家目录
- `-r` 选项是永久删除, 无法恢复
- 如果要求"删除用户", 通常用 `userdel -r`

实际场景:

```
# 场景: 删除用户testuser及其家目录
userdel -r testuser

# 场景: 删除用户tempuser但保留其文件
userdel tempuser
# 家目录/home/tempuser会被保留
```

8.3 组管理

groupadd - 创建组:

```
# 创建新组
groupadd developers

# 创建组并指定GID
groupadd -g 2000 developers

# 参数说明:
# -g = --gid, 指定组ID
```

groupmod - 修改组:

```
# 修改组名
groupmod -n devteam developers

# 参数说明:
# -n = --new-name, 新组名
# devteam = 新名称
# developers = 旧组名
```

groupdel - 删除组：

```
# 删除组
groupdel developers

# ⚠️ 注意：
# 不能删除作为用户主组的组
# 必须先删除用户或修改用户主组
```

SHELL

8.4 密码管理

passwd - 设置密码：

```
# 为当前用户设置密码
passwd

# 为其他用户设置密码（需要root权限）
passwd tom

# 参数说明：
# tom = 目标用户名
```

SHELL

交互式设置流程：

```
Changing password for user tom.
New password:          ← 输入新密码（不显示）
Retype new password:   ← 再次输入密码
passwd: all authentication tokens updated successfully.
```

passwd - 锁定/解锁用户：

```
# 锁定用户（无法登录）
passwd -l tom

# 参数说明：
# -l = --lock, 锁定用户密码

# 解锁用户
passwd -u tom

# 参数说明：
# -u = --unlock, 解锁用户密码
```

SHELL

删除用户密码：

```
# 删除用户密码（用户可无密码登录）
passwd -d tom

# 参数说明：
# -d = --delete, 删除密码
```

SHELL

演示：用户管理实践

演示目标

掌握用户和组的基本管理操作

演示步骤

步骤1：创建用户和组

SHELL

```
# 创建开发组
groupadd developers

# 创建用户并指定组
useradd -G developers alice

# 为用户设置密码
passwd alice
# 输入密码: Alice123!
```

步骤2：查看用户信息

SHELL

```
# 查看用户ID和组
id alice

# 查看用户家目录
ls -la /home/alice/
```

步骤3：修改用户

SHELL

```
# 添加用户到wheel组
usermod -aG wheel alice

# 验证修改
id alice
```

练习：用户管理练习

练习目标

掌握用户和组的创建、修改和删除

练习任务

任务1: 创建项目组

```
# 创建项目组
groupadd projectteam

# 验证组已创建
grep projectteam /etc/group
```

SHELL

任务2: 创建项目用户

```
# 创建用户并加入项目组
useradd -G projectteam bob

# 设置密码
passwd bob
# 输入密码: Bob123!

# 查看用户信息
id bob
```

SHELL

任务3: 修改用户组

```
# 将用户添加到wheel组
usermod -aG wheel bob

# 验证修改
id bob
```

SHELL

任务4: 清理 (可选)

```
# 删除测试用户
userdel -r bob

# 删除测试组
groupdel projectteam
```

SHELL

要点

Day 02 内容

文件管理命令 ★★★★★ 非常重要

硬链接和软链接 ★★ 重要

通配符和扩展 ★★ 重要

man帮助系统 ★ 了解

重定向和管道 ★★★★★ 非常重要

vim编辑器 ★★★★★ 非常重要

用户管理 ★★★★★ 非常重要

组管理 ★★★★★ 非常重要

常用命令清单

文件操作:

```
cp /etc/hosts /tmp/  
mv /tmp/file.txt /tmp/file.bak  
rm /tmp/oldfile.txt  
mkdir -p /tmp/dir1/dir2
```

SHELL

链接操作:

```
ln /etc/hosts /tmp/hosts-hard  
ln -s /etc/hosts /tmp/hosts-soft  
ls -il /tmp/hosts*
```

SHELL

重定向和管道:

```
ls -l /etc > /tmp/output.txt  
ls /etc | grep conf  
find /etc -name passwd 2> /dev/null
```

SHELL

vim操作:

```
vim /tmp/config.txt
# i 进入编辑模式
# Esc 返回命令模式
# :wq 保存退出
```

用户管理：

```
useradd -G wheel user1
passwd user1
usermod -aG group1 user1
userdel -r user1
```

常见任务

1. 查找并复制文件

```
# 查找文件
find /etc -name passwd

# 复制到指定位置
cp /etc/passwd /tmp/
```

2. 创建配置文件

```
# 使用vim创建配置
vim /root/config.txt

# 输入配置内容，保存退出
```

3. 创建用户并设置密码

```
# 创建用户
useradd -G wheel admin

# 设置密码
passwd admin
```

4. 创建软链接

```
# 创建配置文件链接
ln -s /etc/app.conf /tmp/app-link
```

相关练习：

练习名称	对应章节	说明
练习：获得超级用户访问权限	第06章 第02节	使用sudo获取root权限
练习：管理本地用户	第06章 第03节	创建和管理用户账户
练习：管理本地组	第06章 第04节	创建和管理组
总练习：管理本地用户和组	第06章	用户和组管理综合练习

本日总结

关键命令速查

命令	作用	常用选项
cp	复制文件	-r 递归, -a 保留属性
mv	移动/重命名	无
rm	删除文件	-r 递归, -f 强制
ln	创建链接	-s 软链接
man	查看帮助	章节号
> >>	输出重定向	> 覆盖, >> 追加
2>	错误重定向	无
\	管道	无
vim	编辑文件	无
useradd	创建用户	-G 指定组
usermod	修改用户	-aG 添加组
userdel	删除用户	-r 删除家目录
groupadd	创建组	-g 指定GID
passwd	设置密码	-l 锁定, -u 解锁

感谢参与!

下节课见! 🐧